

КАФЕДРА Системы обработки информации и управления

Визовый центр РФ – заявки на визы

Проверил

(Подпись, дата)

А.И. Канев

(И.О.Фамилия)

2026 г.

ВВЕДЕНИЕ

Целью выполнения лабораторных работ является освоение современных технологий веб-разработки, формирование практических навыков проектирования пользовательских интерфейсов и создание полноценных веб-приложений по теме: визовый центр РФ – заявки на визы, услуги – виды виз, заявки – заявки на получение нужной визы.

В процессе работы реализован комплекс лабораторных заданий, направленных на поэтапное изучение:

- основ HTML и CSS с последующей реализацией веб-калькулятора в стилистике сайта визового центра РФ [1];
- принципов интерактивности с использованием JavaScript, включая динамическое обновление интерфейса;
- базовых возможностей платформы Node.js и npm, создание двухстраничного сайта с применением выбранного компонента в рамках заданной тематики;
- написания и интеграции алгоритмов различной сложности на JavaScript в общий проект;
- разработки и тестирования собственного API с поддержкой CRUD-операций и возможностью фильтрации данных.
- взаимодействия с API с использованием как устаревшего механизма XMLHttpRequest, так и современных подходов на основе fetch с применением асинхронного программирования (async/await);
- построения связного интерфейса, в полной мере интегрирующего данные с сервера с возможностью их редактирования и удаления в режиме реального времени.

Таким образом, проект направлен не только на овладение техническими средствами веб-разработки, но и на развитие навыков проектирования интерфейсов в контексте конкретной дизайн-системы — в данном случае, сайта визового центра, а также закрепление современных практик клиент-серверного взаимодействия.

ЛАБОРАТОРНАЯ РАБОТА №1 (Калькулятор HTML/CSS)

- **Задача:** Создание калькулятора. Верстка на HTML, CSS.

В рамках лабораторной работы был разработан калькулятор, содержащий:

- Поле для вывода результата.
- Кнопки операций и чисел.
- Кнопку перехода в github репозиторий.
- **Используемые технологии**
- HTML – для создания структуры страницы.
- CSS – для стилизации элементов.
- **Выполненные задания**

В процессе работы был разработан графический интерфейс калькулятора (Рисунок 1)

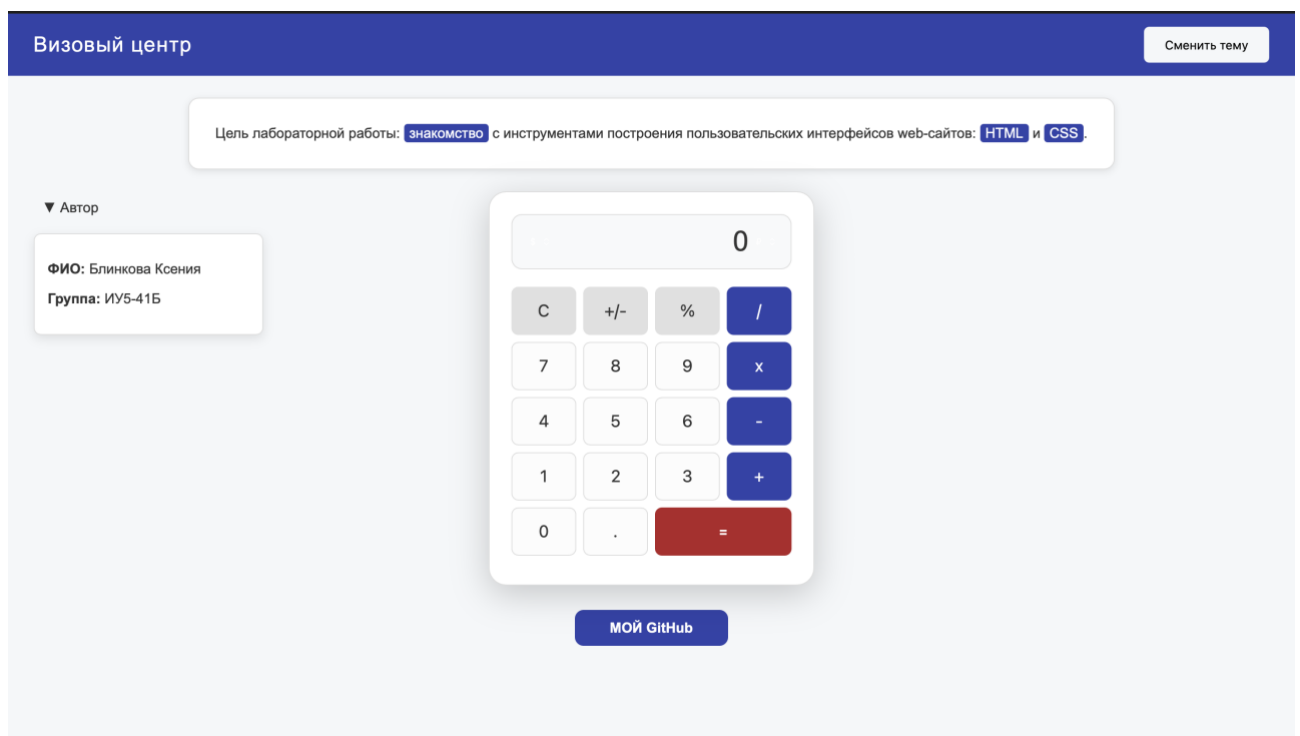


Рисунок 1 - Конечный вид страницы

При разработке интерфейса ключевой задачей было обеспечить визуальную преемственность с официальным стилем единого визового центра [1].

Для этого был проведён анализ оригинального сайта, фрагменты которого представлены далее (**Рисунок 2**, **Рисунок 3**).

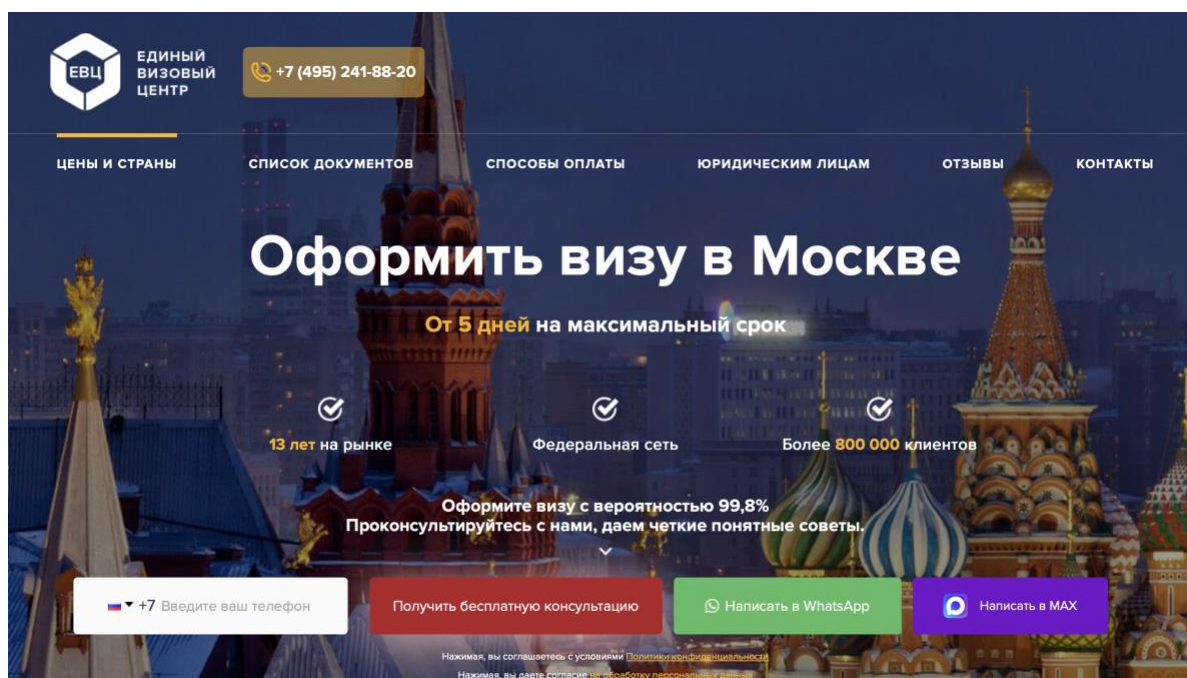


Рисунок 2 - Вид сайта единого визового центра

На **Рисунок 3** выделены преобладающие цветовые акценты интерфейса. Именно эти цвета были проанализированы и отобраны для дальнейшего использования. Детальное описание заимствованной цветовой схемы приведено после **Рисунок 3**.

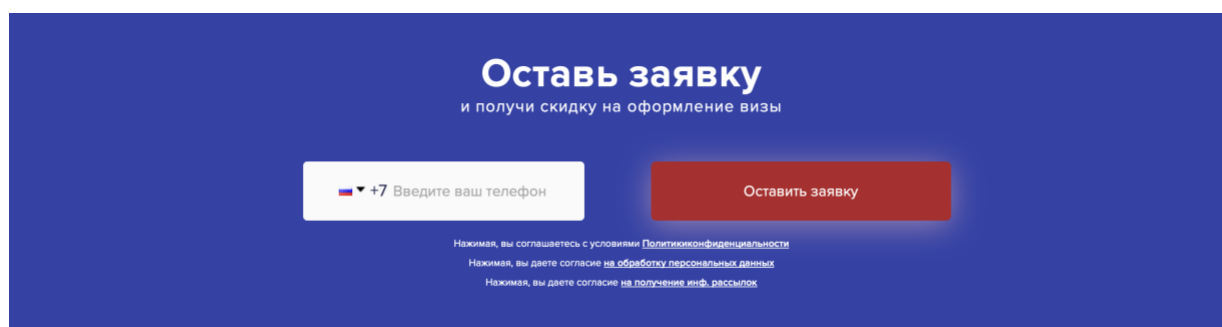


Рисунок 3 - Вид сайта единого визового центра

При разработке дизайна калькулятора из макета визового центра были заимствованы цветовая гамма (синий, красный, белый и серый цвета), стиль

кнопок со скругленными углами и эффектами при наведении, шрифт. Реализация представлена на Рисунок 4

```
.calc-btn {  
    width: 74px;  
    height: 55px;  
    border-radius: 8px;  
    border: 1px solid #dddddd;  
    background: #fcfcfc;  
    color: #333333;  
    font-size: 19px;  
    cursor: pointer;  
}
```

Рисунок 4 - Стиль кнопок

Для создания визуальной связи с официальным сайтом единого визового центра был создан хедер, выполненный в синем цвете (#3242AA). Это решение обеспечивает узнаваемость интерфейса и придает проекту завершенный вид. Реализация представлена на Рисунок 5

```
.main-header {  
    width: 100%;  
    background-color: #3242AA;  
    color: white;  
    padding: 15px 0;  
    display: flex;  
    justify-content: center;  
}
```

Рисунок 5 - Реализация хедера

Среди дополнительных элементов присутствуют кнопка перехода на GitHub, отдельный блок с подробной информацией об авторе, скрытый по умолчанию с возможностью разворачивания через элемент <details>.

ЛАБОРАТОРНАЯ РАБОТА №2 (Калькулятор HTML/CSS/JS)

- **Задача:** Создание калькулятора. Функции на JavaScript.

- Описание проекта

Проект представляет собой веб-калькулятор, реализованный с использованием HTML, CSS и JavaScript (Рисунок 6). В приложении предусмотрены стандартные арифметические операции, однако, помимо базовых вычислений, реализована авторская тематическая функция «Перевести» (calculateVisaEquivalent), специфичная для предметной области «Визовый центр». Данная функция предназначена для мгновенной конвертации вводимых сумм в эквиваленты различных мировых валют.

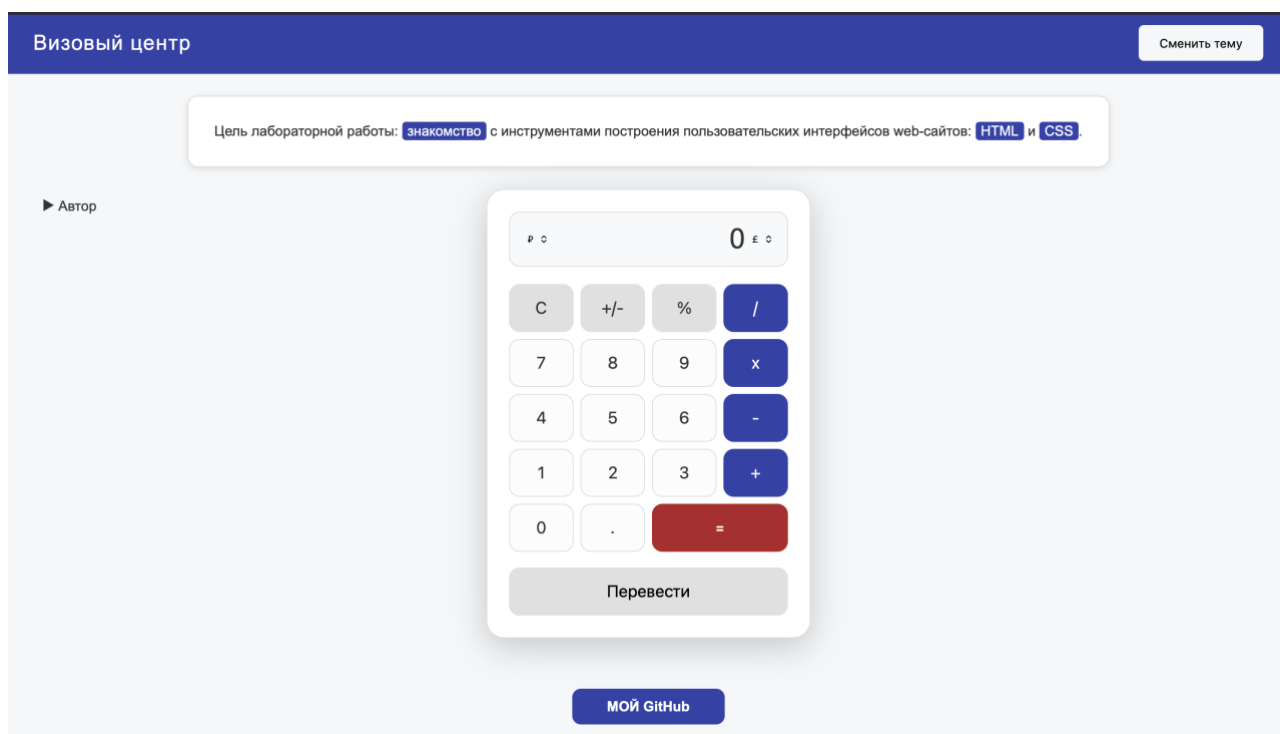


Рисунок 6 - Общий вид калькулятора

Глобальные переменные и инициализация (Рисунок 7)

Для управления состоянием приложения используются переменные, хранящие текущий ввод, предыдущее значение и выбранную операцию. Это позволяет реализовывать сложные цепочки вычислений и корректно отображать данные на дисплее калькулятора.

```
const display = document.getElementById('visa-display');
const fromCurr = document.getElementById('from-currency');
const toCurr = document.getElementById('to-currency');

let currentInput = "";
let previousValue = 0;
let operator = null;
```

Рисунок 7 - Глобальные переменные и инициализация

Реализация арифметических функций (Рисунок 8)

Каждая операция вынесена в отдельную стрелочную функцию. Названия функций адаптированы под предметную область (визовый центр): например, addApplications для сложения заявок или multiplyIssuedVisas для расчета общего количества выданных виз.

```
const addApplications = (a, b) => a + b;
const subtractDenials = (a, b) => a - b;
const multiplyIssuedVisas = (a, b) => a * b;
const divideByConsulates = (a, b) => b !== 0 ? a / b : "Err";
```

Рисунок 8 - Реализация арифметических функций

Авторская функция расчета бюджета (Рисунок 9)

Особое внимание в работе уделено функции calculateVisaEquivalent. Она демонстрирует работу с бизнес-логикой: мгновенную конвертацию введенной суммы в эквиваленты иностранных валют (USD, EUR, GBP). Это необходимо для быстрой оценки стоимости консульских сборов и финансовых гарантий в разных валютных единицах.

```
const calculateVisaEquivalent = () => {
  let amount = parseFloat(currentInput || display.innerText.replace(/[\^d.-]/g, ''));
  if (isNaN(amount)) return;
  const result = (amount * visaExchangeRates[fromCurr.value]) / visaExchangeRates[toCurr.value];
  display.innerText = result.toFixed(2) + " " + visaCurrencySymbols[toCurr.value];
  currentInput = result.toString();
  shouldResetScreen = true;
};
```

Рисунок 9 - реализация функции calculateVisaEquivalent

Динамическая смена тем оформления (рисунок 9)

Для реализации кастомизации интерфейса разработан механизм переключения тем через манипуляцию классами или стилями элементов. При смене темы цвет фона страницы.

```
const themeToggle = document.getElementById('theme-toggle');

themeToggle.addEventListener('click', () => {
  document.body.classList.toggle('dark-theme');
  if (document.body.classList.contains('dark-theme')) {
    themeToggle.innerText = 'Светлая тема';
  } else {
    themeToggle.innerText = 'Сменить тему';
  }
});
```

Рисунок 10 - Реализация смены темы

ЛАБОРАТОРНАЯ РАБОТА №3. Простое веб-приложение. Верстка

- **Задача:** Знакомство с node, npm. Верстка интерфейса с карточками (страница списка с фильтрацией и страница подробнее), данные получать через mock объекты (коллекция). Добавить кнопку добавления (копировать первую карточку), кнопку удаления карточки. В хедере на обеих страницах должна быть кнопка Домой.

- **Описание проекта:** проект представляет собой двухстраничное веб-приложение, разворачиваемое с помощью npm https-server представляющее собой стилизованную по сайту единого визового центра [1] группу кнопок в виде карточек с подписями названий вид виз с возможностью поиска нужной услуги, добавления новой карточки (копирования первой) и удаления. Структура проекта представлена на Рисунок 11

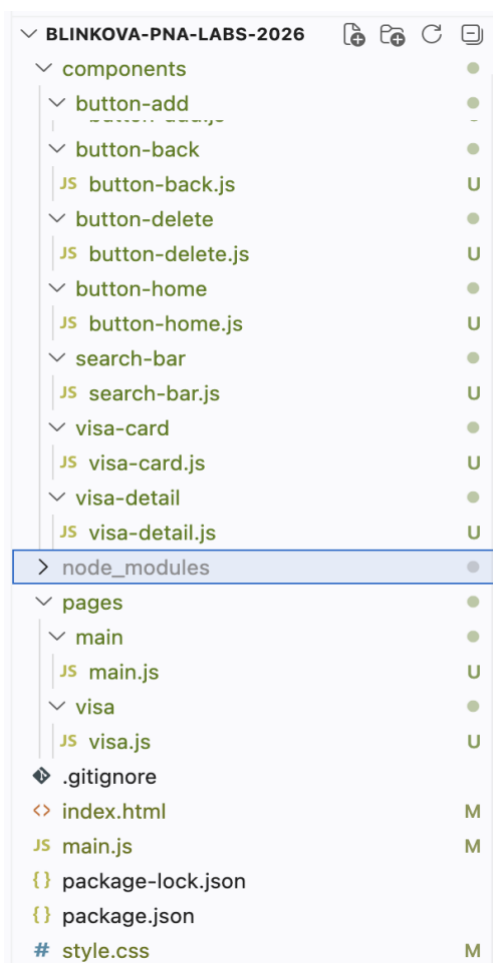


Рисунок 11 - Структура проекта

На главной странице (Рисунок 12) расположены карточки, поиск по названию карточки и кнопки для добавления и удаления карточки. Тема карточек — ВИДЫ ВИЗ.

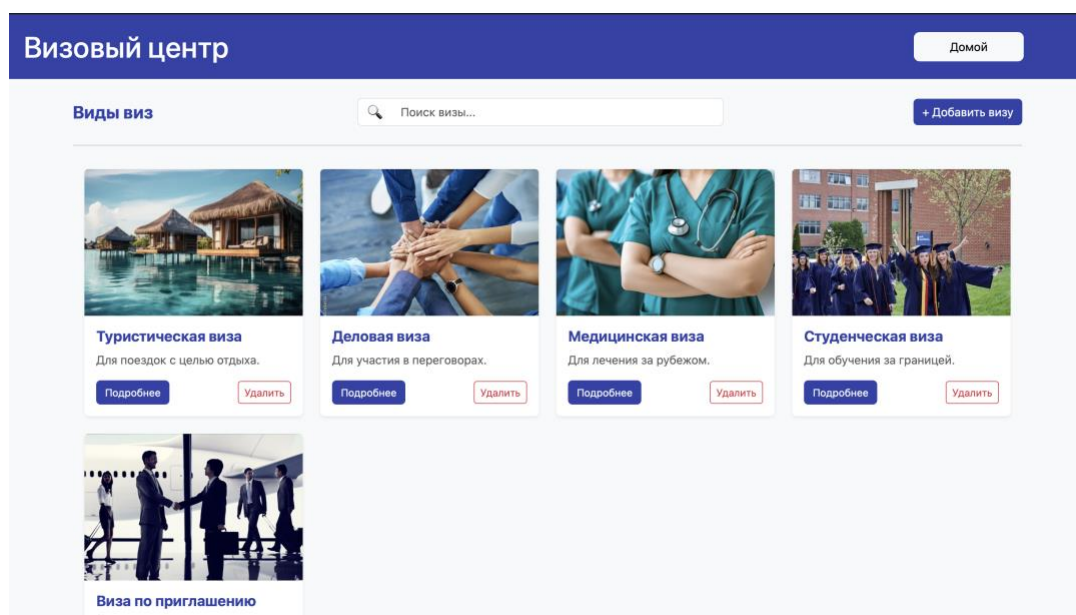


Рисунок 12 - Главная страница

При нажатии на кнопку подробнее переносит на страницу карточки (Рисунок 13) с окном, в котором содержится подробная информация о выбранном виде визы, также инструкция, кнопка “Домой” и кнопка “Назад”.

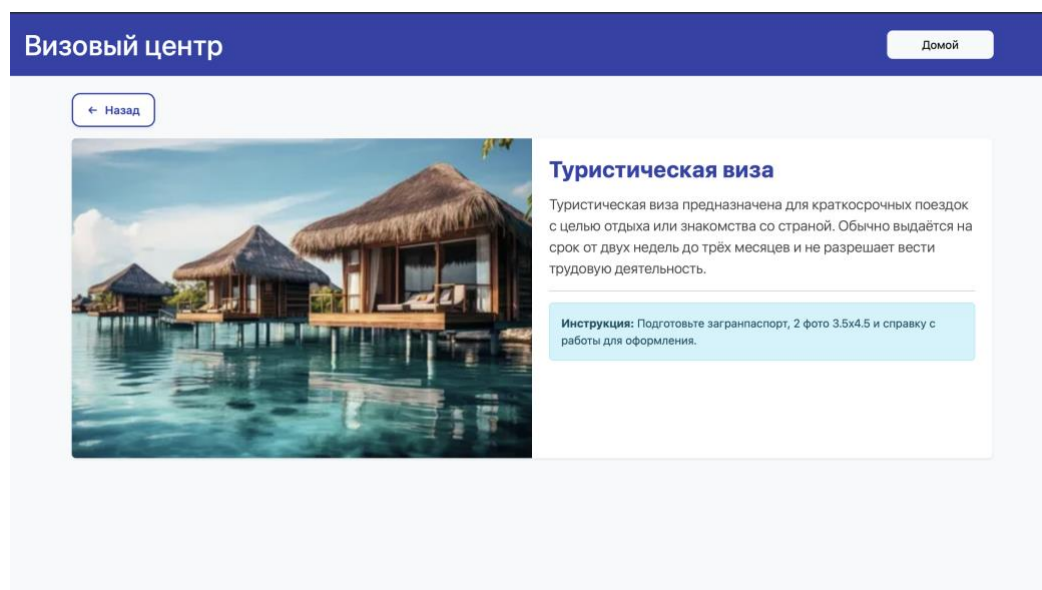


Рисунок 13 - Страница подробнее

Для обеспечения навигации по приложению была разработана кнопка возврата на главную страницу (кнопка “Домой”). Данный элемент является критически важным, так как позволяет пользователю мгновенно сбросить состояние фильтров и вернуться к полному списку виз из любой точки приложения. Реализация приведена на Рисунок 14.

```
const headerAction = document.getElementById('home-button-container');  
  
const homeBtn = new ButtonHome(headerAction);  
homeBtn.render(() => window.renderPage(MainPage));  
  
window.renderPage = (PageClass, ...args) => {  
  root.innerHTML = '';  
  const page = new PageClass(root, ...args);  
  page.render();  
};
```

Рисунок 14 - Реализация программного механизма навигации и функции возврата на главную страницу

Взаимодействие с данными организовано на уровне класса MainPage. Модель данных представлена массивом объектов visaData, где каждый объект содержит уникальный идентификатор, путь к изображению, заголовок и краткое описание услуги. Для манипуляции состоянием интерфейса реализованы следующие механизмы: поиск осуществляется по заголовкам виз, реализован метод addVisa, который создает копию существующего объекта и метод deleteVisa, который производит фильтрацию массива, исключая объект, который выбрал пользователь. Реализация приведена на Рисунок 15.

```

handleSearch(term) {
  const filtered = this.visaData.filter(v =>
    v.title.toLowerCase().includes(term.toLowerCase())
  );
  this.renderCards(filtered);
}

addVisa() {
  const newVisa = {
    ...this.visaData[0],
    id: Date.now(),
    title: `${this.visaData[0].title} (копия)`
  };
  this.visaData.push(newVisa);
  this.renderCards(this.visaData);
}

deleteVisa(id) {
  // Перезаписываем массив, исключая элемент с указанным id
  this.visaData = this.visaData.filter(v => v.id !== id);
  this.renderCards(this.visaData);
}

```

Рисунок 15 - Программная реализация методов фильтрации, добавления и удаления объектов в коллекции данных.

Таким образом, вся логика выполняется за пределами файла главной страницы, и на ней остаются только вызовы соответствующих функций для работы с кнопками, как представлено на рисунке 16.

```

render() {
  this.parent.innerHTML = this.getHTML();
  new SearchBar(document.getElementById('search-placeholder'), (term) => {
    const filtered = this.visaData.filter(v => v.title.toLowerCase().includes(term.toLowerCase()));
    this.renderCards(filtered);
  }).render();
  new ButtonAdd(document.getElementById('add-btn-placeholder')).render(() => this.addVisa());
  this.renderCards(this.visaData);
}

```

Рисунок 16 - Обработчики кнопок на главной странице

ДОМАШНЕЕ ЗАДАНИЕ

- **Задача:** Работа с коллекциями, функциями, классами.

Также на страницы сайта из лабораторной работы №3 были нативно интегрированы задачи в 2 частях из домашней работы в соответствии с вариантом, а также добавлена возможность просмотра 3D-модели оборудования с использованием библиотеки Three.js.

Для подключения модулей библиотеки three и GLTFLoader без использования сборщика (webpack/vite) в корневой файл index.html был добавлен механизм `<script type="importmap">` (Рисунок 17).

```
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
```

Рисунок 17 - Подключение модулей через importmap

На страницу детального просмотра информации о визе добавлен блок рендеринга 3D-модели. Метод `init` инициализирует сцену, настраивает перспективную камеру, освещение и загружает модель в формате `.glb`, автоматически центрируя ее в контейнере (Рисунок 18).

```

// Метод инициализации 3D-сцены в классе PlanetComponent
init() {
  this.scene = new THREE.Scene();
  this.camera = new THREE.PerspectiveCamera(45, this.parent.clientWidth / this.parent.clientHeight, 0.1, 1000);
  this.camera.position.set(2, 1.5, 3);

  this.renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
  this.renderer.setSize(this.parent.clientWidth, this.parent.clientHeight);
  this.parent.appendChild(this.renderer.domElement);

  // Подключение управления мышью
  this.controls = new OrbitControls(this.camera, this.renderer.domElement);
  this.controls.enableDamping = true;

  // Настройка освещения
  this.scene.add(new THREE.AmbientLight(0xffffff, 1));
  const dirLight = new THREE.DirectionalLight(0xffffff, 2);
  dirLight.position.set(5, 5, 5);
  this.scene.add(dirLight);

  // Загрузка модели в формате .glb
  const loader = new GLTFLoader();
  loader.load('../models/planet.glb', (gltf) => {
    this.model = gltf.scene;
    this.scene.add(this.model);
    this.animate();
  });
}

```

Рисунок 18 - Инициализация сцены и загрузка 3D-модели

На Рисунок 19 представлена страница с подробной информации после рендеринга 3D-модели. У модели можно изменять масштаб, а также с помощью кнопок можно изменять положение модели.

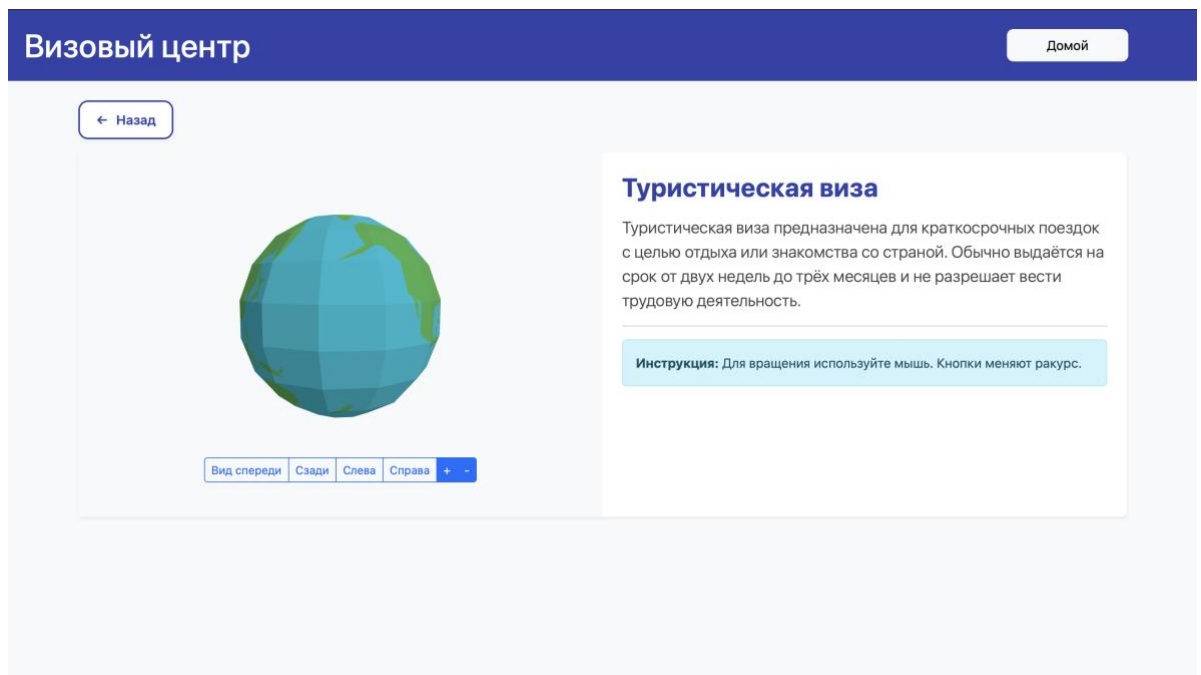


Рисунок 19 - Страница подробнее

Задание 2.3. Дана строка - последовательность, которая может состоять как из 0, так и 1. Необходимо найти максимальную последовательность 1 и вывести ее длину. Реализация приведена на Рисунок 20

```
const str = '1000000111100011111010111101111111';  
  
const maxOnes = Math.max(...str.split('0').map(group => group.length));  
  
console.log(maxOnes);
```

Рисунок 20 - Реализация 2.3

Алгоритм поиска максимальной последовательности единиц (задача 2.3) интегрирован непосредственно в компонент отрисовки карточки VisaCardComponent. Система анализирует строку истории одобрений и отображает пользователю итоговую максимальную серию одобрений (например, "4 из 10"). Это позволяет преобразовывать абстрактные данные в полезную инфографику без участия пользователя. Применение на Рисунок 21.

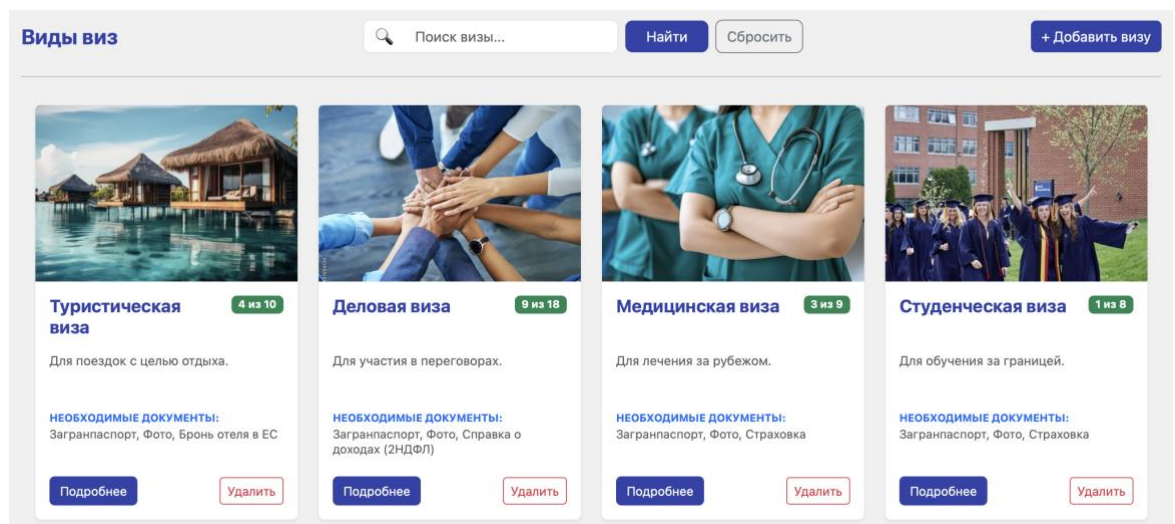


Рисунок 21 - Применение 2.3 в лабораторной №3

Задача 3.1: Напишите функцию merge, которая будет принимать на вход несколько объектов (любое количество), и возвращать единственный объект, содержащий все поля из всех объектов. Если одно и то же поле было в нескольких объектах, необходимо оставить значение, которое встретилось раньше. Реализация задачи представлена на **Error! Reference source not found.**

```

merge(...objects) {
  const result = {};
  let i = 0;

  if (objects.length === 0) return result;

  do {
    const currentObj = objects[i];
    // Перебор ключей текущего объекта
    for (const key in currentObj) {
      // Проверка: берем только собственные свойства и только если ключа еще нет в результате
      if (currentObj.hasOwnProperty(key) && !(key in result)) {
        result[key] = currentObj[key];
      }
    }
    i++;
  } while (i < objects.length);

  return result;
}

```

Рисунок 22 - Реализация задачи 3.1

В информационной системе визового центра данный алгоритм используется для динамического формирования перечня **необходимых документов** для каждой конкретной категории виз.

Логика работы в интерфейсе:

1. Имеется объект `generalDocs` с базовым набором документов, общим для всех (паспорт, фото).
2. Для каждой карточки в базе данных определен объект `extraDocs` с уникальными требованиями (например, «Бронь отеля» или «Справка о доходах»).
3. Метод динамически формирует полный пакет документов для каждой карточки. Он объединяет статический объект с общими документами (паспорт, фото) и объект с дополнительными требованиями, специфичными для конкретной визы.

Применение показано на Рисунок 23.


```

cardElement.querySelector('.merge-btn').onclick = () => {
  const generalDocs = { passport: "Загранпаспорт", photo: "Фото" };
  const countryDocs = data.extraDocs || { insurance: "Страховка" };

  // Слияние объектов
  const fullPackage = this.merge(generalDocs, countryDocs);

  const output = document.getElementById(`merge-output-${data.id}`);
  output.textContent = "Нужно: " + Object.values(fullPackage).join(", ");
};

```

Рисунок 23 - Использование функции из 3.1 в лабораторной №3

Благодаря использованию `merge`, система гарантирует отсутствие дубликатов в списке документов (например, если «Фото» указано в обоих наборах, оно отобразится один раз) и позволяет гибко дополнять требования в зависимости от выбранной страны.

ЛАБОРАТОРНАЯ РАБОТА №4. Реализация собственного API на Node.js

Задача: Реализация собственного API на Node.js. Тестирование через Postman 5 методов: получение списка (в том числе с фильтрацией), получение одной записи, добавление, редактирование, удаление.

Описание проекта

В рамках лабораторной работы было разработано серверное приложение на базе Node.js и микрофреймворка Express.js (использовался JavaScript). Приложение предоставляет полноценный REST API для управления каталогом услуг (карточками). Для персистентности данных используется локальный JSON-файл (stocks.json), который эмулирует работу базы данных.

Архитектура и реализация

Проект имеет слоистую архитектуру, разделяющую логику маршрутизации, обработки запросов и работы с данными: - Routes (visasRouter.js) – определяет конечные точки и связывает их с контроллерами. – Controllers (visasController.js)– отвечает за валидацию входящих данных (например, проверку корректности id или наличия обязательных полей src, title, text) и формирование HTTP-ответов. - Services (visasService.js, fileService.js) – содержит основную бизнес-логику CRUD-операций.

Тестирование API в Postman

Тестирование работоспособности проводилось с помощью инструмента Postman. Сервер обрабатывает запросы по базовому адресу <http://localhost:3005/visas>.

1. **Получение списка всех записей (GET)** Выполнен запрос на получение массива всех доступных карточек. (Рисунок 24)

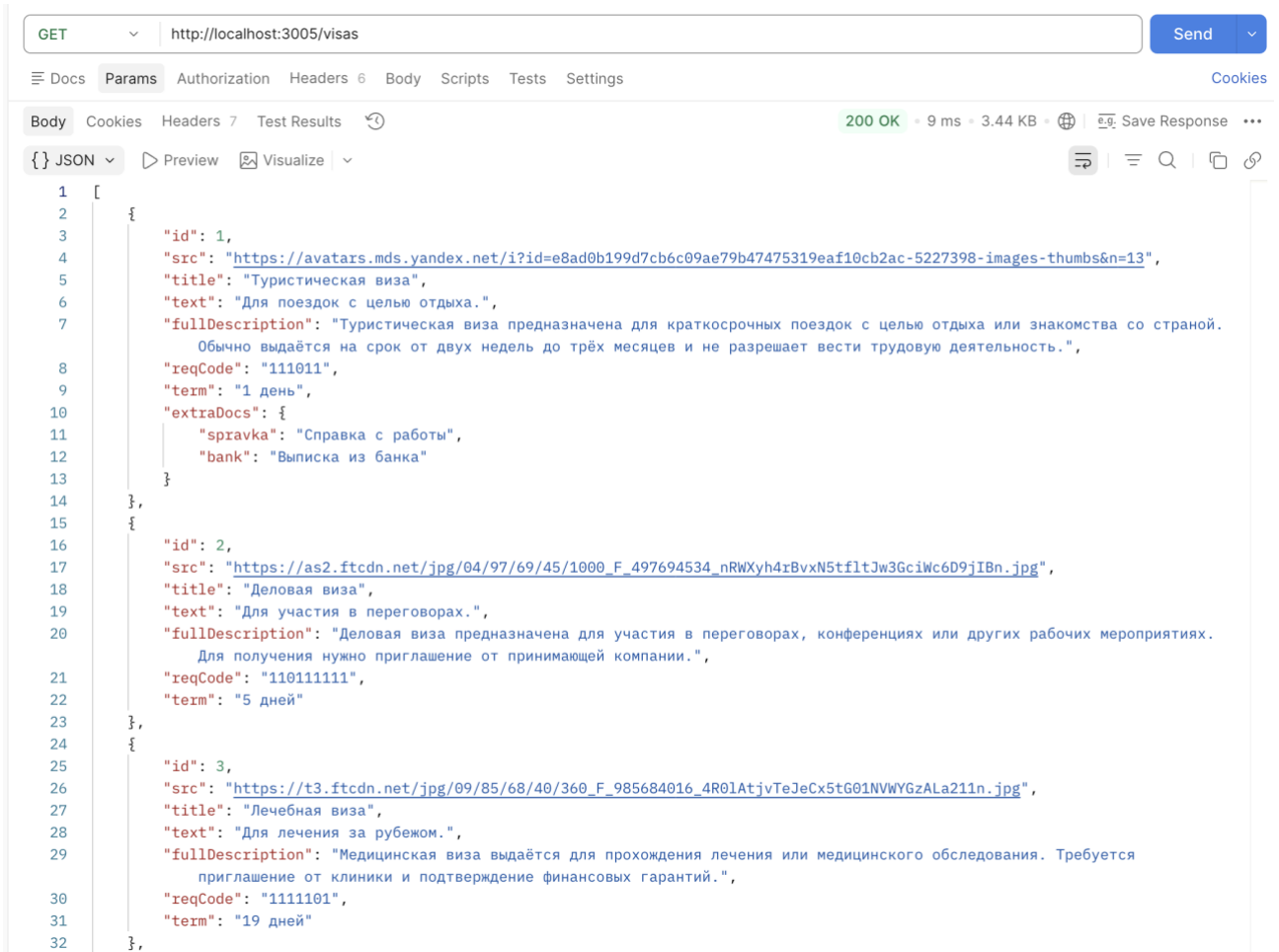


Рисунок 24 - Запрос GET для получения всех записей

2. **Создание новой записи (POST)** В теле запроса (Body -> JSON) переданы поля src, title, text. Сервер автоматически сгенерировал новый id и вернул созданный объект со статусом 201 Created. (Рисунок 25)

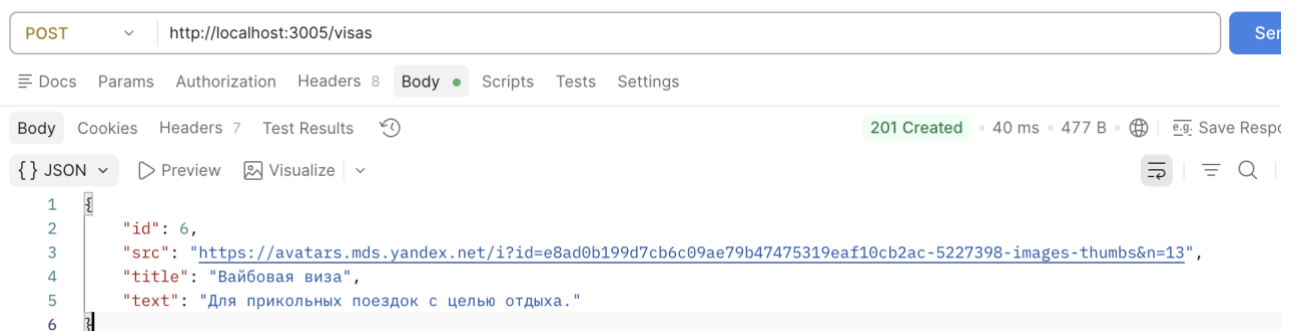


Рисунок 25 - Запрос POST для создания карточки

3. **Получение записи по ID (GET)** Выполнен запрос с указанием конкретного идентификатора в параметрах пути (например, /visas/1). (Рисунок 26)

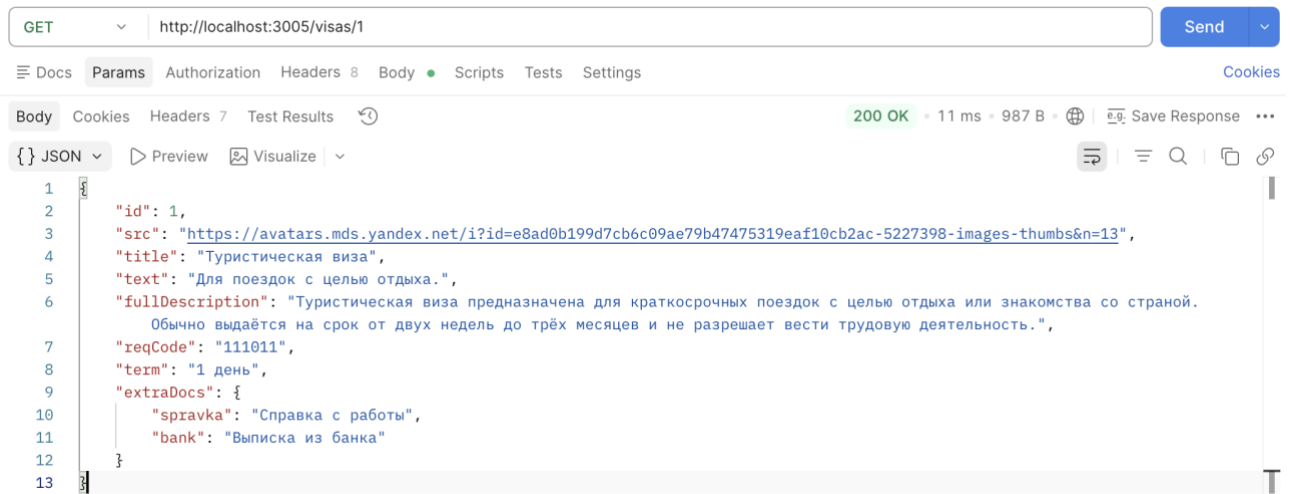


Рисунок 26 - Запрос GET для получения карточки по id

4. **Редактирование записи (PATCH)** Протестировано частичное обновление данных. В теле запроса переданы только те поля, которые необходимо изменить (например, обновлен только title). (Рисунок 27)

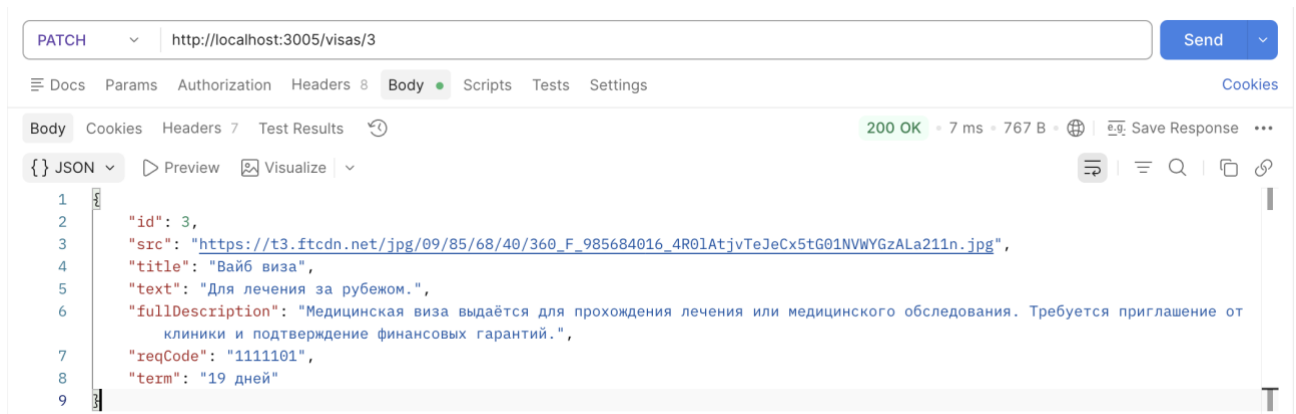


Рисунок 27 - Запрос PATCH для обновления карточки

5. **Фильтрация записей (GET с параметром)** Выполнен GET-запрос с использованием query-параметра ?title=... для проверки логики фильтрации на стороне сервиса. (Рисунок 28)

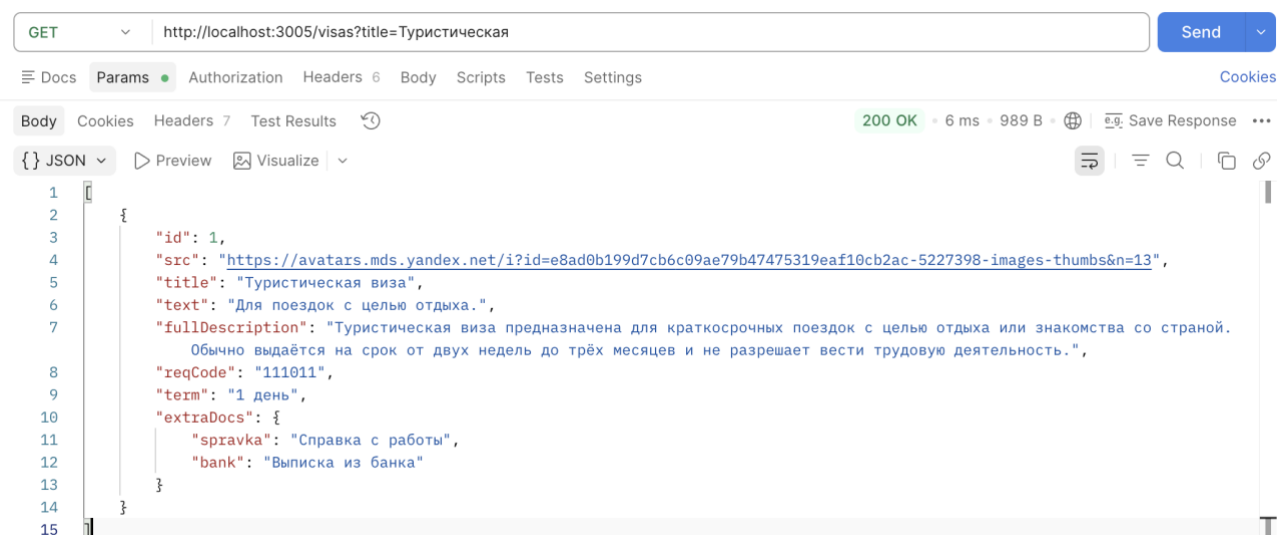


Рисунок 28 - Запрос GET с фильтрацией по title

6. **Удаление записи (DELETE)** Выполнен запрос на удаление существующей карточки по её ID. В случае успешного удаления сервер возвращает статус 204 No Content. (Рисунок 29)

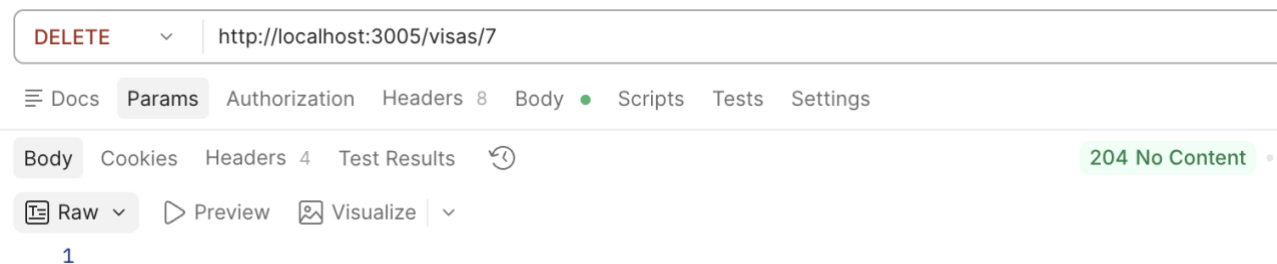


Рисунок 29 - Запрос DELETE для удаления карточки

ЛАБОРАТОРНАЯ РАБОТА №5. XHR запросы.

Продолжение Лабораторной работы 3: добавить страницу добавления/редактирования и соответствующие кнопки, подключение к созданному API бэкенду. Запросы XHR, Cors обойти через расширение браузера CORS Unblock. В ходе выполнения работы ознакомимся с кодом реализации простого взаимодействия с внешним API, получением данных и их выводом в интерфейс пользователя. XMLHttpRequest (или XHR) позволяет делать HTTP-запросы к серверу из браузера без перезагрузки страницы. Несмотря на наличие слова "XML" в названии, с помощью XHR можно работать с любыми типами данных, а не только с XML. С помощью XML можно загружать/скачивать файлы, отслеживать прогресс и многое другое. Перед началом работы с API разберемся с тем, как это будет реализовано в нашем проекте. Сначала создадим еще один слой, где будем хранить все методы работы с API. Текущая структура проекта представлена на **Error! Reference source not found..**

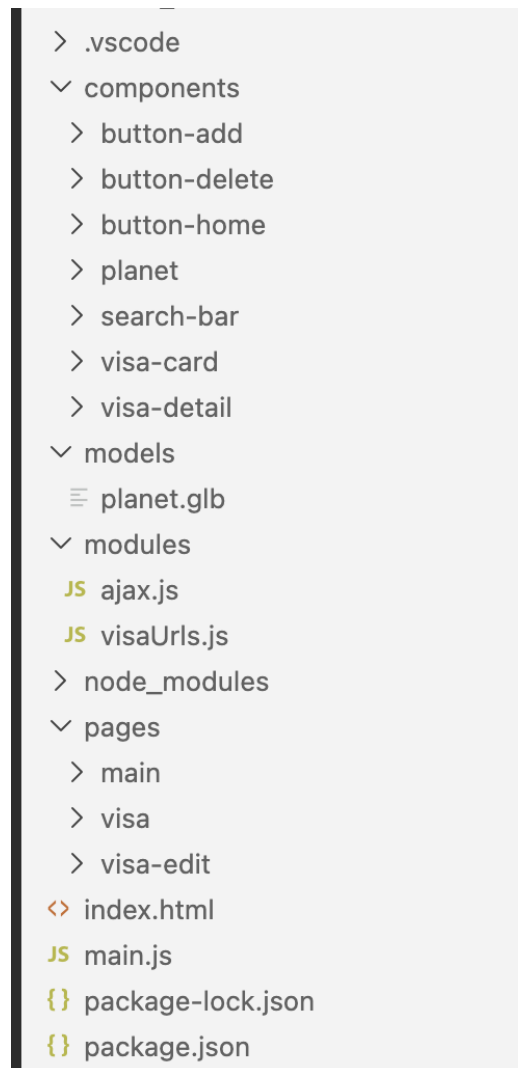


Рисунок 30 - Структура проекта лабораторной №5

Для работы нам понадобятся эндпоинты API, разработанные в предыдущей лабораторной работе. Запустим сервер с помощью команды `npm run start` и убедимся, что он работает и готов слушать запросы (**Error! Reference source not found.**). Сервер запустится и будет доступен по адресу <http://localhost:3005>

```
› kseniablinkova@MacBook-Air-Ksenia Blinkova-PNA-Labs-2026 % npm run start  
  
> blinkova-pna-labs-2026@1.0.0 start  
> node src/index.js  
  
Сервер IU5-41B запущен: http://localhost:3005  
[2026-05-06T18:22:41.093Z] GET /visas  
[2026-05-06T18:22:42.123Z] GET /visas  
█
```

Рисунок 31 - Запуск сервера с помощью npm

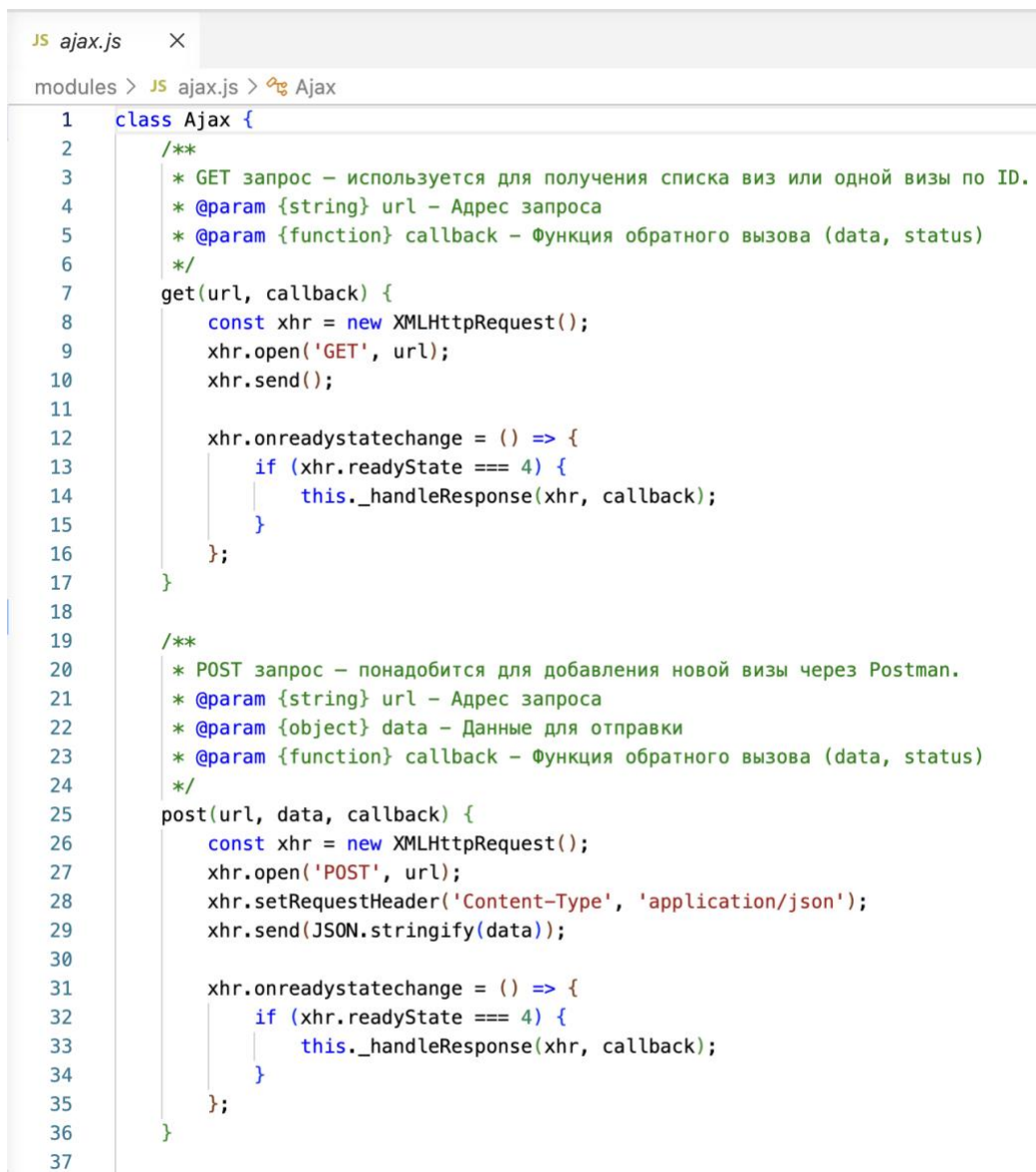
Объявим нужные эндпоинты для карточек в отдельном файле, чтобы можно было использовать их в нескольких местах и легко менять базовый URL. Базовый URL — <http://localhost:3005>, каждый запрос будет выполняться по /visas. Создаем файл modules/visaUrls.js (**Error! Reference source not found.**).

```
JS visaUrls.js ●
modules > JS visaUrls.js > ...
1  class VisaUrls {
2      constructor() {
3          this.baseUrl = 'http://localhost:3005';
4      }
5
6      getVisas() {
7          return `${this.baseUrl}/visas`;
8      }
9
10     getVisaById(id) {
11         return `${this.baseUrl}/visas/${id}`;
12     }
13
14     createVisa() {
15         return `${this.baseUrl}/visas`;
16     }
17
18     removeVisaById(id) {
19         return `${this.baseUrl}/visas/${id}`;
20     }
21
22     updateVisaById(id) {
23         return `${this.baseUrl}/visas/${id}`;
24     }
25 }
26
27 export const visaUrls = new VisaUrls();
```

Рисунок 32 - Эндпоинты

Для взаимодействия с API создаётся файл modules/ajax.js, содержащий класс с методами выполнения XHR-запросов и обработки ответов от сервера. На главной странице (pages/main/main.js) данные о доступных визах загружаются

через асинхронный запрос к API с помощью метода `getData` и отрисовываются в интерфейсе с помощью метода `renderCards`.

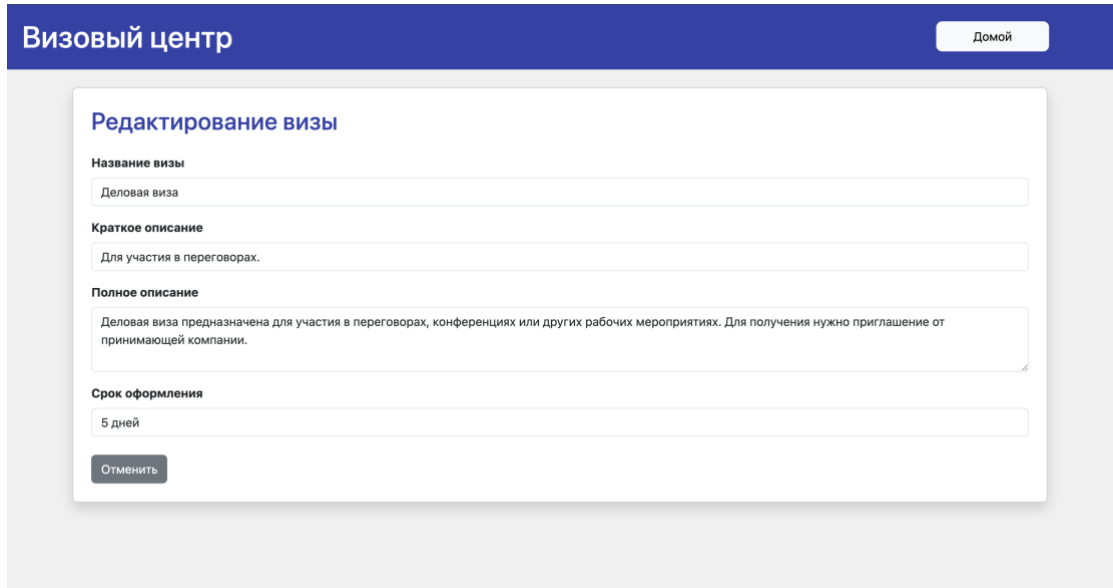


```
1 class Ajax {
2   /**
3    * GET запрос – используется для получения списка виз или одной визы по ID.
4    * @param {string} url – Адрес запроса
5    * @param {function} callback – Функция обратного вызова (data, status)
6    */
7   get(url, callback) {
8     const xhr = new XMLHttpRequest();
9     xhr.open('GET', url);
10    xhr.send();
11
12    xhr.onreadystatechange = () => {
13      if (xhr.readyState === 4) {
14        this._handleResponse(xhr, callback);
15      }
16    };
17  }
18
19  /**
20   * POST запрос – понадобится для добавления новой визы через Postman.
21   * @param {string} url – Адрес запроса
22   * @param {object} data – Данные для отправки
23   * @param {function} callback – Функция обратного вызова (data, status)
24   */
25  post(url, data, callback) {
26    const xhr = new XMLHttpRequest();
27    xhr.open('POST', url);
28    xhr.setRequestHeader('Content-Type', 'application/json');
29    xhr.send(JSON.stringify(data));
30
31    xhr.onreadystatechange = () => {
32      if (xhr.readyState === 4) {
33        this._handleResponse(xhr, callback);
34      }
35    };
36  }
37}
```

Рисунок 33 - Обработка запросов в ajax

На странице редактирования (`pages/visa-edit/visa-edit.js`) реализовано получение данных конкретной визы с помощью GET-запроса через модуль `ajax`. Метод `render` отображает индикатор загрузки до момента получения ответа от сервера, после чего метод `getHTML` формирует форму редактирования, подставляя полученные данные (название, краткое и полное описание, срок оформления) в соответствующие поля ввода (Рисунок 34). Взаимодействие с

интерфейсом включает обработку нажатия кнопки «Отменить», которая возвращает пользователя на главную страницу. Все сетевые взаимодействия и корректность получения данных из API отслеживаются через вкладку Network в инструментах разработчика DevTools.



The screenshot displays a web application titled 'Визовый центр' (Visa Center) with a blue header. A 'Домой' (Home) button is located in the top right corner. The main content area features a form titled 'Редактирование визы' (Edit Visa). The form contains four input fields: 'Название визы' (Visa Name) with the value 'Деловая виза' (Business Visa), 'Краткое описание' (Brief Description) with the value 'Для участия в переговорах.' (For participation in negotiations.), 'Полное описание' (Full Description) with the value 'Деловая виза предназначена для участия в переговорах, конференциях или других рабочих мероприятиях. Для получения нужно приглашение от принимающей компании.' (Business visa is intended for participation in negotiations, conferences or other work events. To obtain it, you need an invitation from the host company.), and 'Срок оформления' (Processing Time) with the value '5 дней' (5 days). At the bottom of the form is an 'Отменить' (Cancel) button.

Рисунок 34 - Страница редактирования

Стоит отметить, что в текущем состоянии со стороны клиента сайт представлен множеством составных частей. Структура сайта, видимая во вкладке sources, представлена на Рисунок 35.

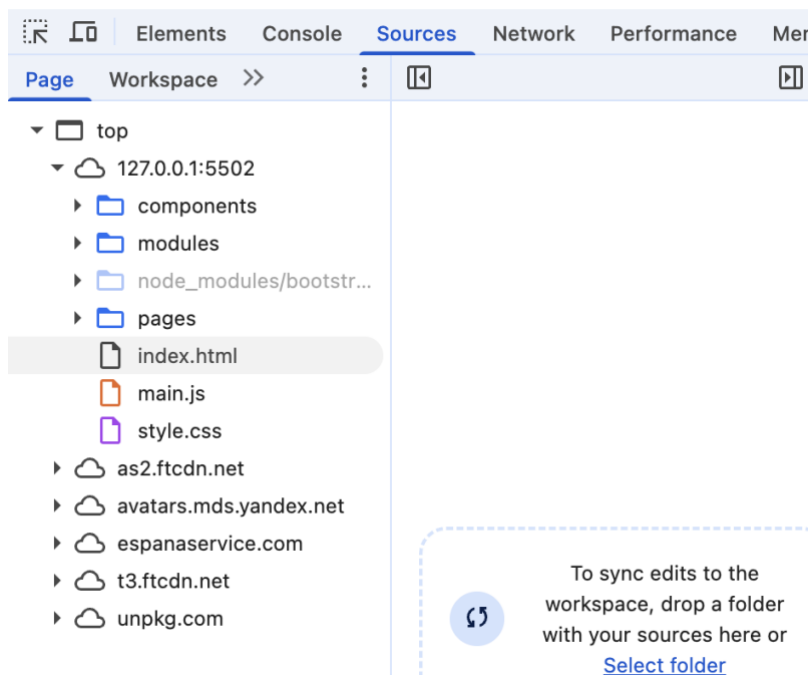


Рисунок 35 - Структура сайта

Без какого-либо способа обхода CORS-политики, однако, никакие запросы работать не будут. Для решения этой проблемы в данной лабораторной работе используется браузерное расширение CORS Unblock. Используемые настройки расширения приведены на Рисунок 36.

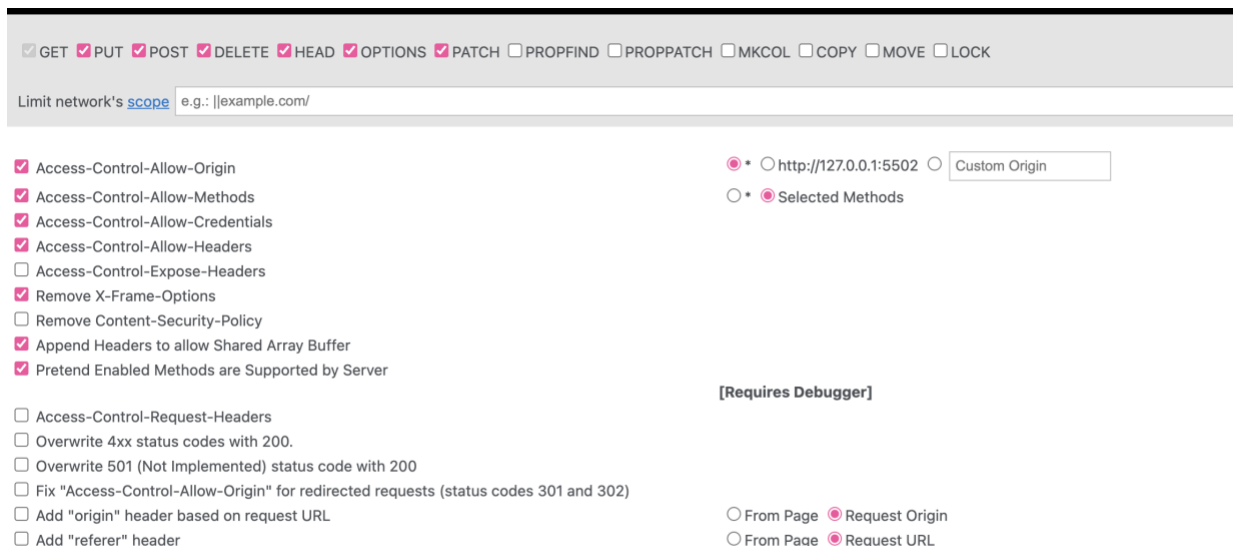


Рисунок 36 - Настройки расширения CORS Unblock

ЛАБОРАТОРНАЯ РАБОТА №6. Простое веб-приложение. Async/Await, замена XMLHttpRequest на Fetch. Подключение к бэкенду 4 лабораторной работы.

В данной лабораторной работе была реализована задача взаимодействия фронтенда и бэкенда с использованием современных технологий для работы с внешним API. Цель заключалась в замене устаревшего XMLHttpRequest на более современный метод fetch, а также в реализации полного цикла CRUD-операций для работы с продуктами. Для этого были созданы три страницы: главная страница для отображения списка услуг и страница редактирования с возможностью добавления, удаления и страница просмотра детальной информации об услуге. В структуру проекта 4 лабораторной работы была добавлена 1 папка. Структура лабораторной работы №4 представлена на Рисунок 37.

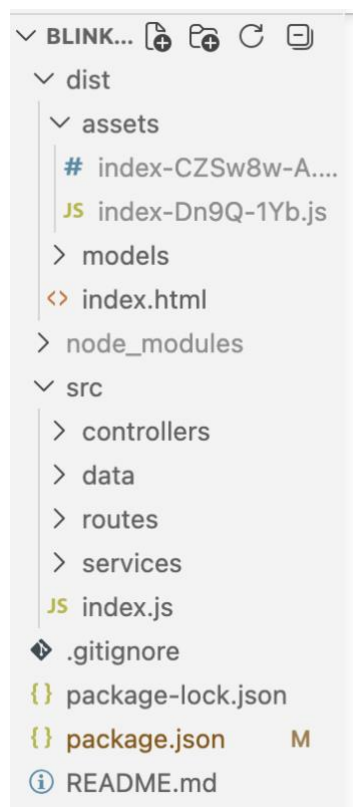


Рисунок 37 - Структура лабораторной работы №4

Бэкенд был реализован на Express.js в лабораторной работе №4, где были созданы необходимые модули, контроллеры и сервисы для работы с данными. Основной файл конфигурации бэкенда находится в файле index.js, а логика работы с файлом данных реализована в fileService.js. Для хранения данных использовался JSON-файл visas.json, который эмулирует базу данных. Методы для работы с карточками продуктов (создание, чтение, обновление и удаление) были реализованы в visasService.js, а маршруты API описаны в visasController.js. Основной JavaScript-код, реализующий логику взаимодействия с бэкендом, написан с применением модульной системы и собирается в единый бандл с помощью утилиты Vite. Результат сборки (статические файлы, оптимизированные скрипты и стили) располагается в директории dist, которая раздается сервером как статическая.

Корневым файлом фронтенд-части является index.html, который служит точкой входа для приложения. Взаимодействие с API полностью переведено на современный метод fetch, который возвращает промисы (Promises), что позволяет использовать синтаксис async/await для более чистой обработки асинхронных операций. Примеры использования fetch реализованы в модуле страницы редактирования (pages/visa-edit/visa-edit.js), где выполняются запросы GET для получения данных визы, а также в методах удаления и загрузки списка на главной странице. (Рисунок 38)

```
async saveData() {
  const saveBtn = document.getElementById('save-btn');
  const btnText = document.getElementById('save-btn-text');
  const spinner = document.getElementById('save-btn-spinner');
  const msgContainer = document.getElementById('message-container');

  // Собираем данные из полей ввода
  const updatedData = {
    title: document.getElementById('title-input').value,
    text: document.getElementById('text-input').value,
    fullDescription: document.getElementById('desc-input').value,
    term: document.getElementById('term-input').value
  };

  // Включаем индикацию загрузки
  saveBtn.disabled = true;
  btnText.textContent = "Сохранение...";
  spinner.classList.remove('d-none');
  msgContainer.innerHTML = ''; // Очищаем старые сообщения
}
```

Рисунок 38 - Пример запроса PATCH

Для обеспечения корректной работы приложения были выполнены следующие шаги: настройка сервиса для работы с файлами в файле `fileService.js`, настройка маршрутов в файле `visasRoutes.js`, реализация контроллера в файле `visasController.js`, а также настройка главного приложения в файле `index.js`.

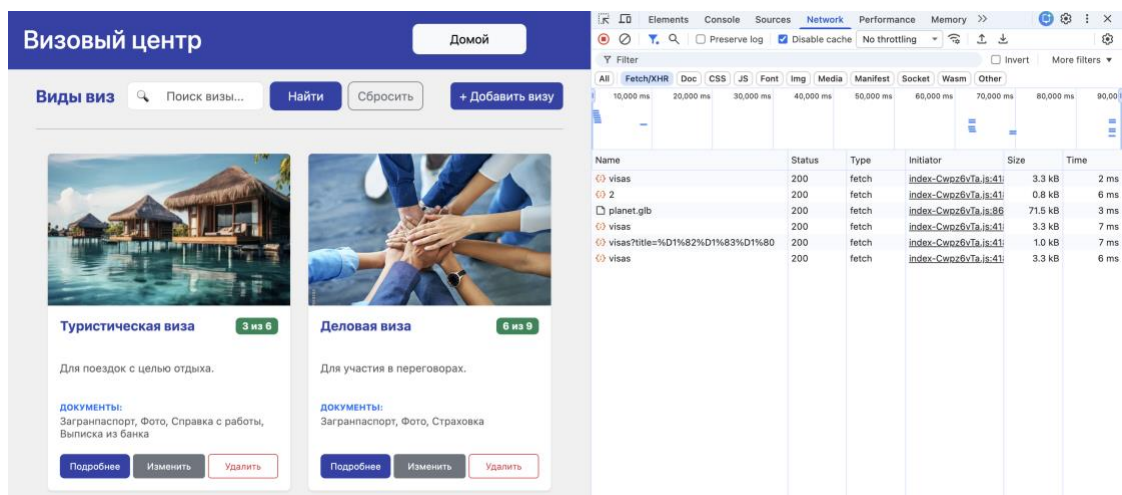


Рисунок 39 - fetch запросы

Стоит отметить, что после сборки фронтенда проекта в `dist`, структура сайта со стороны клиента сильно упростилась. Структура сайта, видимая во вкладке `sources`, представлена на Рисунок 40

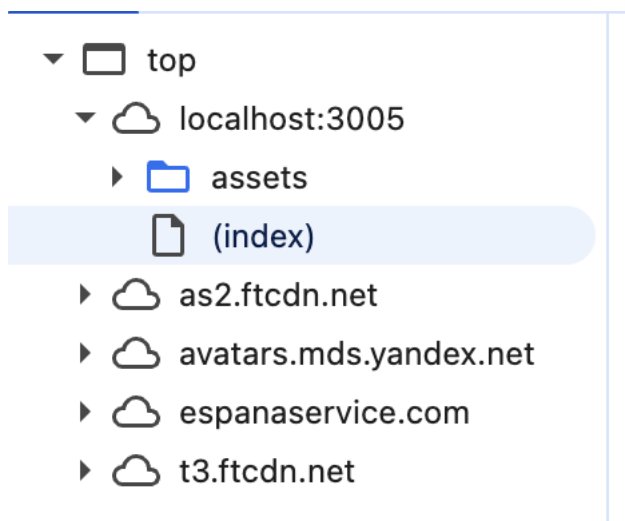


Рисунок 40 - Структура сайта во вкладке sources

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторных работ реализована система для предметной области «Визовый центр РФ — заявки на визы».

- В рамках первой лабораторной работы был создан стилизованный калькулятор, выдержанный в стиле официального сайта визового центра. Это позволило освоить основы HTML и CSS .

- Во второй лабораторной работе была добавлена интерактивность с помощью JavaScript, что обеспечило не только базовые арифметические операции, но и тематическую функцию конвертации валюты.

- В третьей лабораторной работе было разработано приложение-галерея, реализованное с помощью Bootstrap, которое позволило организовать каталог визовых услуг с возможностью добавления и удаления карточек виз.

- В домашнем задании произошла интеграция авторских алгоритмов (поиск максимальной последовательности единиц для анализа истории одобрений и функция слияния merge для объединения объектов с общими и дополнительными документами), которая позволила адаптировать абстрактные вычислительные задачи под прикладную логику визового центра.

- В четвёртой лабораторной работе был создан REST API на Node.js, который обеспечил полноценную серверную поддержку CRUD-операций для управления визовыми заявками и услугами, что было подтверждено тестированием через Postman.

- В пятой лабораторной работе была произведена интеграция API из предыдущей работы в приложение из лабораторной работы 3, что позволило реализовать редактирование, добавление и удаление визовых карточек без перезагрузки страницы.

- В шестой работе xhr запросы были заменены на fetch, а клиентская часть приложения была объединена с серверной с помощью Vite.

Вывод. Разработанная система полностью соответствует заявленной предметной области «Визовый центр РФ — заявки на визы».

Репозиторий проекта: https://github.com/KseniaBlinkova/visa_center.git

ЛИТЕРАТУРА

1.Сайт единый визовый центр – официальная домашняя страница [Электронный ресурс] – URL: <https://единый-визовый-центр.рф/msk/>.

2.Лекции по курсу XML-технологии и методические указания по выполнению лабораторных работ [Электронный ресурс]. – URL: <https://github.com/iu5git/JavaScript/blob/main>